



PDF Download
3770581.pdf
03 April 2026
Total Citations: 2
Total Downloads: 354

 Latest updates: <https://dl.acm.org/doi/10.1145/3770581>

RESEARCH-ARTICLE

Integrating Various Software Artifacts for Better LLM-based Bug Localization and Program Repair

QIONG FENG, Nanjing University of Science and Technology, Nanjing, Jiangsu, China

XIAOTIAN MA, Nanjing University of Science and Technology, Nanjing, Jiangsu, China

JIAYI SHENG, Nanjing University of Science and Technology, Nanjing, Jiangsu, China

ZIYUAN FENG, Nanjing University of Science and Technology, Nanjing, Jiangsu, China

WEI SONG, Nanjing University of Science and Technology, Nanjing, Jiangsu, China

PENG LIANG, Wuhan University, Wuhan, Hubei, China

Open Access Support provided by:

Nanjing University of Science and Technology

Wuhan University

Published: 03 October 2025
Accepted: 28 September 2025
Revised: 25 June 2025
Received: 05 December 2024

[Citation in BibTeX format](#)

Integrating Various Software Artifacts for Better LLM-based Bug Localization and Program Repair

QIONG FENG, Nanjing University of Science and Technology; State Key Lab. for Novel Software Technology, Nanjing University, China

XIAOTIAN MA, Nanjing University of Science and Technology, China

JIAYI SHENG, Nanjing University of Science and Technology, China

ZIYUAN FENG, Nanjing University of Science and Technology, China

WEI SONG, Nanjing University of Science and Technology, China

PENG LIANG*, School of Computer Science, Wuhan University, China

LLMs have garnered considerable attention for their potential to streamline Automated Program Repair (APR). LLM-based approaches can either insert the correct code using an infilling-style technique or directly generate patches when provided with buggy methods, aiming for plausible patches to pass all tests. However, most of LLM-based APR methods rely on a single type of software information, such as issue descriptions or error stack traces, without fully leveraging a combination of diverse software artifacts. Human developers, in contrast, often use a range of information — such as debugging data, issue discussions, and error stack traces — to diagnose and fix bugs. Despite this, many LLM-based approaches do not explore which specific types of software information best assist in localizing and repairing software bugs. Addressing this gap is crucial for advancing LLM-based APR techniques.

To investigate this and mimic the way human developers fix bugs, we propose DEVLoRE (short for DEveloper Localization and REpair). In this framework, LLMs first use issue content (description and discussion) and stack error traces to localize buggy methods, then rely on debug information in buggy methods and issue content and stack error to localize buggy lines and generate valid patches. We evaluated the effectiveness of issue content, error stack traces, and debugging information in bug localization and automatic program repair. Our results show that while issue content and error stack is particularly effective in assisting LLMs with fault localization and program repair respectively, different types of software artifacts complement each other in addressing various bugs. By incorporating these three types of artifacts and using the Defects4J v2.0 dataset for evaluation, DEVLoRE successfully localizes 49.3% of single-method bugs and generates 56.0% plausible patches. Additionally, DEVLoRE can localize 47.6% of non-single-method bugs and generates 14.5% plausible patches. Moreover, our framework streamlines the end-to-end process from buggy source code to a complete repair, and achieves a 39.7% and 17.1% of single-method and non-single-method bug repair rate, outperforming current state-of-the-art APR methods. Furthermore, we re-implemented and evaluated our framework, demonstrating its effectiveness in resolving 9 unique issues compared to other state-of-the-art frameworks using the same or more advanced models on SWE-bench Lite. We also discussed whether a

*Corresponding author

Authors' Contact Information: Qiong Feng, Nanjing University of Science and Technology; State Key Lab. for Novel Software Technology, Nanjing University, Nanjing, China, qiongfeng@njust.edu.cn; Xiaotian Ma, Nanjing University of Science and Technology, Nanjing, China, xyzboom@njust.edu.cn; Jiayi Sheng, Nanjing University of Science and Technology, Nanjing, China, shengjiayi@njust.edu.cn; Ziyuan Feng, Nanjing University of Science and Technology, Nanjing, China, azumaseren@njust.edu.cn; Wei Song, Nanjing University of Science and Technology, Nanjing, China, wsong@njust.edu.cn; Peng Liang, School of Computer Science, Wuhan University, Wuhan, China, liangp@whu.edu.cn.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, or post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2025 Copyright held by the owner/author(s).

ACM 1557-7392/2025/10-ART

<https://doi.org/10.1145/3770581>

leading framework for Python code can be directly applied to Java code, or vice versa. The source code and experimental results of this work for replication are available at <https://github.com/XYZboom/DEVLoRe>.

CCS Concepts: • **Software and its engineering** → **Software maintenance tools**; **Software verification and validation**.

Additional Key Words and Phrases: Large Language Model, Automatic Program Repair, Fault Localization

1 INTRODUCTION

Automatic Program Repair (APR) streamlines the process of identifying and correcting code defects, significantly reducing the time and effort required for manual bug fixing [22]. Traditional APR techniques employ various methods, including template-based approaches [11, 19, 21, 26, 28, 30] and neural machine translation (NMT) [9, 14, 25, 52], to generate potential patches that are both syntactically valid and semantically meaningful. Although these methods can produce correct patches for certain bugs, they also have notable limitations. NMT models rely heavily on bug-fixing training data, making them unable to generate patches for new or unseen types of bugs. On the other hand, template-based approaches suffer from a limited set of templates and struggle to address more complex, nontrivial bug fixes [43].

Recent studies have explored the use of LLMs for APR, either by having LLMs fill in the correct code in buggy methods using an infilling-style technique or by directly generating patches when provided with buggy methods [13, 24, 36, 39–44, 50]. The initial results demonstrate the ability of LLMs to correctly repair real-world bugs, including those that were previously unrepairable by existing APR approaches. One of the best-performing frameworks is Agentless [40], which feeds an LLM with only issue description and can automatically solve GitHub issues. These promising outcomes highlight the potential of LLMs to develop more effective APR methods.

However, there are still two major **limitations** that need to be addressed:

- Current LLM-based approaches do not fully incorporate various types of software artifacts. Most LLMs rely on just one or two kinds of software artifacts, such as issue descriptions or code structure, while other important artifacts are underutilized. For instance, debugging information, which is a critical tool for human developers in diagnosing and resolving bugs, is often not fully leveraged.
- While various LLM-based approaches make use of different software artifacts, such as issue descriptions and error stack traces, it remains unclear which specific type of information most effectively aids LLMs in localizing and automatically repairing software bugs.

To address these two limitations and further explore the ability of LLMs to localize and fix software bugs (bug, defect, and fault are used interchangeably in this paper), we propose feeding LLMs different types of software artifacts to determine which information best leverages their capabilities in bug localizing and fixing. The rationale behind this approach is that human developers typically do not rely on a single type of information when localizing and fixing software bugs. Instead, they combine various sources of information in software development, such as issue descriptions, proof of concept (PoC), stack traces, discussions in issues, and more. Based on developers' experience, having more information helps to better understand the root cause of bugs, ultimately leading to more effective bug localization and fixes.

To achieve this, we propose the DEVLoRE framework, which asks LLMs to mimic human developers for bug localization and program repair. Along with this framework, we design two tools to extract executed methods in failed test cases and debugging information of buggy methods. In this framework, we feed the chosen LLM with three types of software artifacts: issue content (including issue description and discussion), error stack trace, and debug information. Then, using the well-known Defects4J dataset [17], we evaluate how these different types of software artifacts contribute to effectively localizing and fixing bugs. Our experiment results demonstrate that issue content is the most effective indicator for buggy method's localization, achieving 43.6% for localizing single-method bugs and 40.6% for localizing non-single-method bugs. Stack error trace proves to be the best indicator for single-method bug fixing, with a precision of 27.0%. Additionally, our findings highlight that different types of

software information can complement each other in the process of localizing and fixing bugs. By combining issue content and error stack trace, we achieve a state-of-the-art performance of 49.3% in localizing the single-method bugs. Furthermore, incorporating issue content, error stack trace, and debugging information results in a fix rate of 43.1% for single-method bugs.

As our DEVLoRE framework does not specify that the localization and fix of buggy methods should be singular, it can localize and fix bugs across multiple methods (non-single-method bugs). For example, by combining all three software artifacts, DEVLoRE can fix 9.4% of non-single-method bugs with provided buggy locations. Moreover, similar to Agentless [40], DEVLoRE relies on LLMs throughout the entire end-to-end process of fault localization and program repair. It can successfully fix 28.0% of single-method bugs and 11.2% of non-single-method bugs when combining all three artifacts. To the best of our knowledge, our approach outperforms current state-of-the-art methods.

To summarize, our **contributions** in this paper are as follows:

- (1) To the best of our knowledge, this work is the first to compare different software artifacts in assisting LLMs' ability to perform both fault localization and program repair. The results can help developers understand LLMs' potential when provided with a variety of software artifacts.
- (2) We propose a simple and lightweight framework that leverages LLMs to conduct an end-to-end process for bug localization and program repair. Accompanying this framework we design a strict input/output prompt. The DEVLoRE framework demonstrates a strong ability to localize and fix more software bugs in less time and at a lower cost, compared to current state-of-the-art methods on the Defects4J dataset.
- (3) We evaluated our framework on SWE-bench Lite and demonstrated DEVLoRE's effectiveness in resolving 9 unique issues compared to other state-of-the-art frameworks using the same or more advanced models. We further discussed whether a leading framework for Python code can be directly applied to Java code, or vice versa.

The paper is structured as follows: Section 2 discusses the motivation of using various software artifacts, Section 3 outlines the proposed approach, Section 4 introduces the experiment setup and the research questions, Sections 5 and 6 present and discuss the results respectively, Section 7 reviews the related work, and Section 8 concludes this work with future directions.

2 THREE MOTIVATING EXAMPLES

Our assumption is that LLM-based program repair tools have the potential to be much more effective if they can leverage a variety of software artifacts, such as code snippets, version history, documentation, and even testing outputs. By providing LLMs with a rich set of contextual data, they can understand and localize bugs more accurately and offer more precise fixes. This assumption is based on a human software engineer's daily practice. When a human developer tries to fix a bug, they would examine various resources such as the issue descriptions, error stack, debugging information, and test cases, until a solution is identified. However, currently most LLM-based repair approaches underutilize or use limited software artifacts. Here, we provide three examples to demonstrate the motivation of this work and the ability of LLMs to localize and fix bugs when given access to different types of software artifacts.

2.1 Debugging Information to the Rescue

The Lang 1b bug involves the *createNumber* method, which takes a string as its parameter and returns a number represented by that string. While studying the Lang 1b bug, we noticed that the newly added failing test case is the input "0x80000000" for the *createNumber* method. When this test case is provided, the buggy *createNumber* method incorrectly treats the input string as an integer, when it should be a long integer instead. The code snippet in Figure 1 shows a portion of the buggy *createNumber* method in Lang 1b. The local variable

hexDigits represents the valid hexadecimal digits. As human software developers, we can quickly recognize that at Line 471, the condition should be “*hexDigits* > 8 || (*hexDigits* == 8 && *firstSigDigit* > ‘7’)” (*firstSigDigit* refers to the first valid hexadecimal digit).

```

466         if (pfxLen > 0) { // we have a hex number
467             final int hexDigits = str.length() - pfxLen;
468             if (hexDigits > 16) { // too many for Long
469                 return createBigInteger(str);
470             }
471             if (hexDigits > 8) { // too many for an int
472                 return createLong(str);
473             }
474             return createInteger(str);
475         }

```

Fig. 1. Code snippet for Lang 1b

For this bug, we provided the LLM with Lang 1b’s issue content (<https://issues.apache.org/jira/browse/LANG-747>) and the stack error message shown in Figure 2, but both could not help the LLM generate a plausible fix or near-to-correct patch. This is because the leap required in reasoning is significant, and the buggy method is quite long. Neither of the information provided above could pinpoint the buggy line precisely, let alone

Commons Lang / LANG-747

NumberUtils does not handle Long Hex numbers

issue content

without proof of concept

Details

Type: Bug
Priority: Major
Affects Version/s: None
Component/s: lang.math.*
Labels: None

Status: CLOSED
Resolution: Fixed
Fix Version/s: 3.2

Description

NumberUtils.createLong() does not handle hex numbers, but createInteger() handles hex and octal. This seems odd.
NumberUtils.createNumber() assumes that hex numbers can only be Integer.
Again, why not handle bigger Hex numbers?

debugging information

★

ClassName: lang3.math.NumberUtils
MethodNameAndLine:createNumber:468

```
{
  "hexDigits": 8
}
```

error stack

without code info

```

java.lang.NumberFormatException: For input string: "80000000"
    at java.lang.NumberFormatException.forInputString(NumberFormatException.java:65)
    at java.lang.Integer.parseInt(Integer.java:583)
    at java.lang.Integer.valueOf(Integer.java:740)
    at java.lang.Integer.decode(Integer.java:1197)
    at org.apache.commons.lang3.math.NumberUtils.createInteger(NumberUtils.java:684)
    at org.apache.commons.lang3.math.NumberUtils.createNumber(NumberUtils.java:474)
    at org.apache.commons.lang3.math.NumberUtilsTest.TestLang747(NumberUtilsTest.java:256)

```

Fig. 2. Issue content, error stack and debug info of Lang 1b

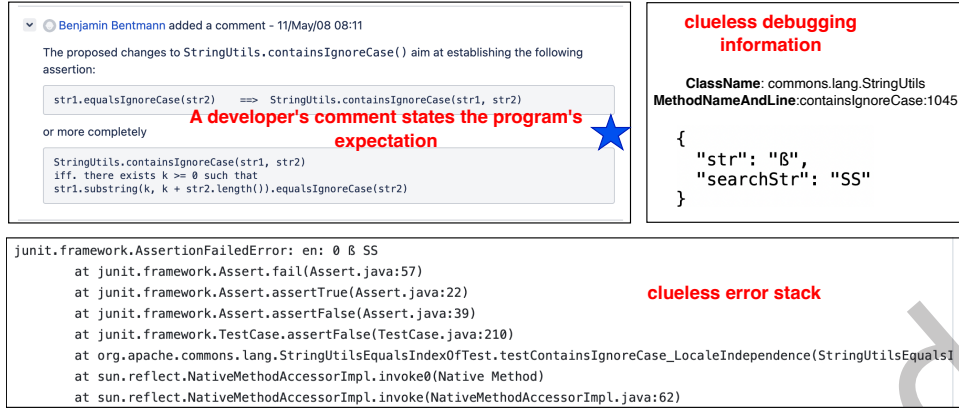


Fig. 3. Issue content, error stack and debug info for Lang 40b

generate a plausible fix. To address this, we introduced debugging information to help the LLM understand the necessary changes. One piece of debug information we provided was a series of variable-value pairs, *commons.lang3.math.NumberUtils:createNumber:468 {hexDigits:8}*. This indicates that the code is about to execute Line 468 and that the value of *hexDigits* is 8 for the given test case. This enables the LLM to understand why the code on Line 474 was executed instead of Line 471. In one of the responses, the LLM suggested changing Line 468 to *if (hexDigits > 16 || (hexDigits == 8 && str.charAt(2) >= '8'))*, which, although not completely accurate, demonstrates that the provided debug information has a positive impact on fixing the bug.

2.2 Issue Content to the Rescue

Sometimes, the discussions and Proof of Concept (PoC) included in the issue content can help the LLM better understand the expectations for bug repair. Figure 3 shows a developer's comment in the issue content, which states the expected behavior of the program (<https://issues.apache.org/jira/browse/LANG-432>). The original code attempted to convert both input strings to uppercase using `String.toUpperCase()` and then return the result of invoking the `contains` method. However, `String.toUpperCase()` is locale-sensitive, which makes it unsuitable for case-insensitive comparisons. For example, the character `0x00DF` represents “ß” in Unicode. If we apply `String.toUpperCase()` to this character, it becomes “SS” in the Turkey locale. Consequently, comparing “ss” with “ß” would result in an equal match. Therefore, in this case, we should not use `String.toUpperCase()` but instead compare the characters individually.

The debug information, *org.apache.commons.lang.StringUtils:containsIgnoreCase:1045 {“str”:“ß”, “searchStr”:“SS”}*, and the error stack shown in Figure 3 merely re-display the test cases and do not clearly highlight the relationship between the character `0x00DF` and ‘SS’. As a result, neither the error stack trace nor the debug information provides any useful clues about the bug, and LLMs cannot generate a plausible fix when provided with either the stack message or the debug information alone. However, with the description and expected behavior outlined in the issue content, GPT-4o is able to generate a correct patch.

2.3 Error Stack to the Rescue

When a bug triggers an exception, as opposed to merely being an unexpected behavior, the error stack trace generated at the point where the exception occurs can be helpful in diagnosing and fixing the error. Stack trace

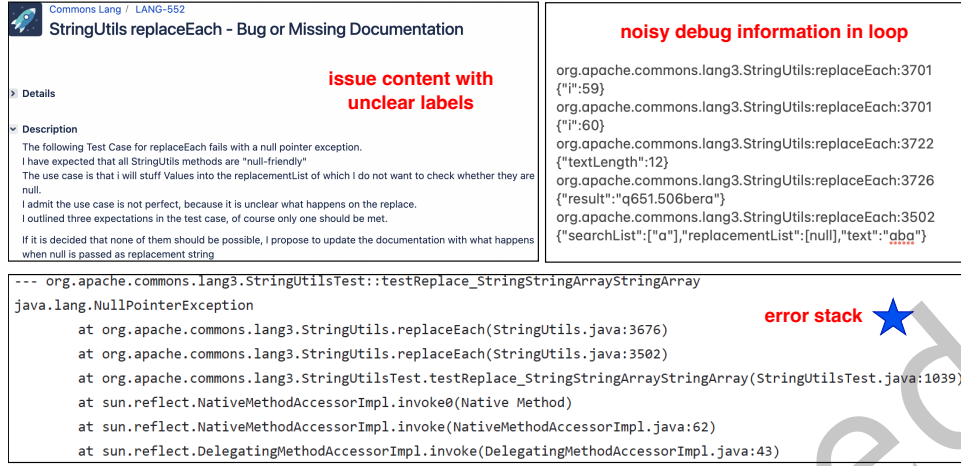


Fig. 4. Issue content, error stack and debug info for Lang 39b

```
diff --git a/src/java/org/apache/commons/lang3/StringUtils.java b/src/java/org/apache/commons/lang3/StringUtils.java
index 14563aa6..2d12a357 100644
--- a/src/java/org/apache/commons/lang3/StringUtils.java
+++ b/src/java/org/apache/commons/lang3/StringUtils.java
@@ -3673,7 +3673,8 @@ public class StringUtils {

    // count the replacement text elements that are larger than their corresponding text being replaced
    for (int i = 0; i < searchList.length; i++) {
-        int greater = replacementList[i].length() - searchList[i].length();
+        int greater = (replacementList[i] != null ? replacementList[i].length() : 0) - searchList[i].length();
+
        if (greater > 0) {
            increase += 3 * greater; // assume 3 matches
        }
    }
}
```

Fig. 5. A patch generated by GPT-4o-mini for Lang 39b

provides a detailed record of the sequence of method calls that led to an exception, making it easier to pinpoint the exact location in the code where the bug arises.

For example, in Lang 39b shown in Figure 4, the developer forgot to check for *null* values in the elements of the input parameters *replacementList* and *searchList*, which can lead to a *NullPointerException* (NPE). However, if we do not inform the LLM about the occurrence of an NPE in the error stack, the LLM will not be able to identify where *null* detection is necessary. The stack trace clearly indicates that the NPE occurs on Line 3676 in the class *StringUtils*. By providing this stack information to the LLM, it is able to generate a repair, as shown in Figure 5.

2.4 Observations From the Above Three Examples

From these three examples, we can see that issue content, error stack traces, and debugging information can complement each other in bug detection and fixing. Each type of artifact provides a different level of context, and when combined, they create a more comprehensive understanding of the bug. Issue content offers the high-level

description and expectations for the fix, error stack traces pinpoint the exact location of the failure, and debugging information provides granular details about the variable states and flow of execution. By incorporating these diverse artifacts, the LLM's repair process becomes more holistic.

3 APPROACH

As shown in Figure 6, our approach utilizes LLMs in two distinct steps: bug localization and program repair. Along with these two steps, we developed two dynamic tools — MethodRecorder and DebugRecorder — to assist LLMs in handling bug localization and fix. MethodRecorder tracks the methods executed during the failing test case(s), while DebugRecorder is designed to extract debug information from the buggy method(s). Debug information is stored in a list, where each entry includes the method name, line number, variable names, and the JSON-formatted content of these variables. Table 1 shows the prompts used in our process. The first, second, and third rows correspond to the prompts used for the tasks of localizing buggy methods, localizing buggy lines, and generating patches, respectively. The first two rows of prompts are used for the bug localization task and the last row of prompt is used for the program repair task. For each task, we provide the LLM with <GENERAL TASK PROMPT> + <INPUT PROMPT> + <EXPECTED OUTPUT PROMPT> to receive the response from the LLM.

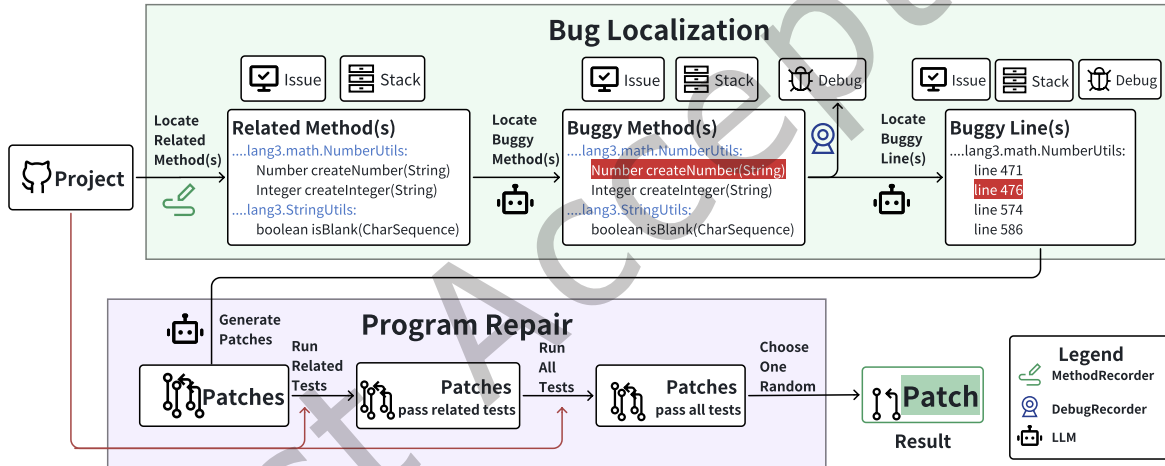


Fig. 6. Our approach for bug localization and program repair

3.1 Localize Buggy Method(s)

We emulate the behavior of human developers, who use errors in stack traces and proof of concept (PoC) from issue content to localize buggy methods. To this end, we apply MethodRecorder to trace the methods invoked during the execution of failing test cases. We then collect the signatures of these invoked methods and prompt the LLM to identify the buggy method(s). In this step, we also provide the LLM with any available error stack traces and issue content, including general description of the issue and developers' discussion/comment under this issue. We choose not to use debugging information because it is typically gathered at runtime and involves instrumentation (see *java.lang.instrument.Instrumentation* [3]), which can be computationally expensive. In developers' daily practice, they set just a few of breakpoints and collect important information about variables.

Table 1. Prompts used in localizing buggy methods, buggy lines and generating patches

GENERAL TASK PROMPT	INPUT PROMPT	EXPECTED OUTPUT PROMPT
You are a Software Engineer. Review the following skeleton of classes, test case(s), and exception that occurs when doing the test. Provide a set of locations that need to be edited to fix the bug. The locations must be specified as method names or field names.	<pre> ### Skeleton of Classes ### {related_methods} {error_stack} {issue_content} </pre>	Please localize class name and method names or field names that need to be edited. Examples: <pre> path.to.ClassA::methodA path.to.ClassA::methodB path.to.ClassB::methodA </pre>
You are a Software Engineer. Review the following skeleton of classes, test case(s), and exception that occurs when doing the test. Provide a set of locations that need to be edited to fix the bug. The locations must be specified as line number in class.	<pre> ### Skeleton of Classes ### {buggy_methods} {error_stack} {issue_content} {debugging_info} </pre>	Please localize class name and line number that need to be edited. Examples: <pre> path.to.ClassA line: 20 line: 45 line: 46 line: 47 </pre>
You are a Software Engineer. Review the following methods and(or) fields of classes, test case(s), and exception that occurs when doing the test. Try to fix the bug.	<pre> ### Skeleton of Classes ### {buggy_methods} Possible bug locations (for your reference only) ### {buggy_lines} {error_stack} {issue_content} {debugging_info} </pre>	Please generate *SEARCH/REPLACE* edits to fix the bug based on the info given above. Every *SEARCH/REPLACE* edit must use this format: 1. The file path 2. The start of search block: «««< SEARCH 3. A contiguous chunk of lines to search in the existing source code 4. The dividing line: ===== 5. The lines to replace into the source code 6. The end of the replace block: >>>> REPLACE

But setting breakpoints automatically is not practical. Therefore, before localizing buggy method(s), minimizing instrumentation and debugging information is more efficient.

3.2 Localize Buggy Line(s)

Once the buggy method(s) have been localized in the previous step, we utilize DebugRecorder to capture detailed information about variable names and their values within the identified buggy method(s). The rationale for introducing debugging information at this stage is that it provides a more granular view, which is particularly beneficial for pinpointing the specific lines responsible for the bug and for understanding the internal state of variables. The debugging information with variables' value is often unnecessary during the initial localization step, but becomes crucial when the focus shifts to identifying precise faults within a known problematic method.

In this process, we gather debugging information as a list of variable-value pairs that reflect the state at specific lines of code within the buggy method(s). This allows the LLM to understand the conditions and data flows

that may contribute to the bug. Combined with the issue content, such as descriptions and discussions, along with stack traces, this debugging information is fed into the LLM along with the body of the buggy method(s). Together, these resources assist the LLM in localizing the exact line(s) within the method(s) where the bug manifests, providing the context needed to understand and resolve the bug effectively.

3.3 Patch Generation

For patch generation, we provide all relevant information to maximize the LLM’s ability to produce an accurate repair. This includes the localized buggy line(s) and method(s) identified in previous steps, as well as the complete body of the buggy method(s). Additionally, we provide supplementary materials such as the issue content, which may contain descriptions and discussions about the bug, the stack trace from the error, and detailed debugging information captured at runtime. By supplying this comprehensive context, we enable the LLM to better understand the specific code behavior that led to the error and to use this knowledge to generate a more targeted and effective patch for the bug. For each bug, we use the LLM to generate multiple potential patches in *.diff* format, which can be directly applied to the buggy code.

3.4 Patch Validation

We apply the patches generated in the previous step to the buggy code and test them to evaluate their effectiveness. If the patch passes the initial failed tests, we proceed to pass all tests in case that it incur any regression. If it passed all the tests, then it is considered a plausible fix. To facilitate this process, we utilize the Defects4J framework to run the tests and verify the patches.

To be consistent with existing studies [24, 53], if the patch is semantically equivalent to the original patch provided by developers, it is considered a correct patch. As part of this validation process, the patch undergoes manual inspection and cross-checking by two experienced developers to ensure its correctness. This step is crucial to verify that the patch not only passes all automated tests but also aligns with the intended behavior from a developer’s perspective. The manual inspection by developers serves as a final quality control step to ensure that the patch addresses the bug correctly and does not introduce new bugs.

4 EXPERIMENT SETUP

4.1 Dataset

We used the well-established Defects4J benchmark [17] for our experiments, specifically leveraging both version 1.2 and version 2.0. Defects4J v1.2 contains 391 bugs from six real-world projects (note that there are 395 bugs in v1.2, but due to the update to Java 8, four bugs can no longer be reproduced), while v2.0 includes an additional 444 bugs from 11 real-world projects. We chose Defects4J benchmark for two main reasons. First, it offers a diverse set of software artifacts — such as issue URLs, error stacks, and test cases — that are well-suited for our proposed ARP framework and help maximize its capabilities. Second, Defects4J benchmark has been widely used in prior research, which facilitates a fair and meaningful comparison of our approach.

4.2 Parameters in Experiments and Baseline Selection

4.2.1 Parameters in Experiments: Due to the extremely long debugging information for some bugs recorded by DebugRecorder, along with issue content and error stack, which exceeds the token limit of most LLMs, we opted for the GPT-4 series model, known for its ability to handle long token sequences with a 128k token context window. To manage costs, we selected the cost-effective **GPT-4o-mini** model, also with a 128k token context window, and used the default settings (*Temperature*=0.5, *Top-p*=1). For the experimental setup, before automating the entire process, we manually used a client with access to multiple LLMs and provided artifact information for 10 random selected Defects4J bugs to evaluate the feasibility of our approach and to select appropriate models—considering

factors such as low cost, a large token context window, and minimal hallucination rate. During this step, we found that the GPT-3.5 series exhibited a high rate of hallucination when generating patches, and we excluded it from consideration. We also determined the appropriate format for the artifacts to be fed into the LLM. For example, we pruned the debug information such as only recording variables that have changed, cropping long arrays and lists to minimize token size. Additionally, we instructed the LLM to generate *SEARCH/REPLACE* edits when producing patches, in order to reduce hallucination rate, as adopted by SOTA approaches [27, 32, 40, 51]. After selecting the model, prompts and artifacts format, we used this preliminary process to determine the optimal number of LLM calls at each step: localizing buggy methods, localizing buggy lines, and generating patches. The rationale is that increasing the number of LLM calls beyond a certain point does not lead to significant performance gains. Based on these manual experiments, we finalized the settings of the parameters. For buggy method localization, we call the LLM only once. For buggy line localization, we call the LLM 10 times, collecting 10 responses, each identifying potential buggy lines. We then filter out duplicate responses (i.e., identical buggy line locations). Finally, for each unique response, we ask the LLM to generate patches three times, ensuring that multiple potential fixes are considered.

4.2.2 Baseline Selections: We selected state-of-the-art LLM-based fault localization and program repair frameworks on the Defects4J dataset as our baselines for two reasons. First, using the same Defects4J dataset allows us to directly compare our results with those reported in the baseline papers. Second, we focus on LLM-based approaches to better evaluate our framework, which is also LLM-based and leveraging a set of software artifacts.

4.3 Research Questions

In this work, we aim to answer the following research questions (RQs) for evaluating the effectiveness of various software artifacts in assisting the chosen LLM in bug localization and APR task.

- **RQ1. Which software artifacts can better assist LLMs in localizing buggy methods and buggy lines when provided with source code?** This RQ aims to identify the specific types of software artifacts - issue content, debugging information and error stack trace - that enhance the ability of LLMs to accurately pinpoint methods and code that contain bugs.
- **RQ2. Which software artifacts can better assist LLMs in generating plausible patches when provided with buggy methods?** By answering this RQ, we can identify the specific types of software artifacts in assisting LLMs to produce plausible patches which can pass all unit tests.
- **RQ3. What is the overall performance of DEVLoRE in bug localization and program repair?** Unlike some LLM-based approaches that focus on either fault localization or program repair while relying on traditional methods such as spectrum-based approaches [5] for fault localization [24, 41], DEVLoRE supports an end-to-end bug identification and program repair workflow. This RQ aims to evaluate the overall performance of DEVLoRE from the initial input of a code repository to the final output of a complete repair.

5 RESULTS

5.1 RQ1. Buggy Method and Buggy Lines Localization from Code Repository

5.1.1 Localizing Buggy Method(s). We first use our MethodRecorder tool, which runs the failing tests and records the signatures of the methods that have been executed to narrow down the scope of buggy methods. Then we provide the LLM with the signatures of the executed methods, along with issue content (including the issue description and discussion, which we crawled from each bug's issue URL) and the error stack trace corresponding to the failing test case, as supplied by the Defects4J framework. In this step, we choose not to generate the debugging information. The main reason is that the debugging information can be too long for LLMs to process

effectively before the buggy method(s) have been localized. Using the prompt specified in Table 1, we ask the LLM to output the signatures of buggy method(s). To evaluate what kinds of software artifacts can better assist the LLM in localizing buggy methods, we conduct four separate experiments by feeding the LLM with (1) only executed method signatures, (2) executed method signatures and issue content, (3) executed method signatures and error stack trace, and (4) executed method signatures, issue content and error stack trace.

Table 2. Effectiveness of buggy method localization in Defects4J v2.0 with different software artifacts

	Single Method	Non-Single
—	135/486=27.8%	77/320=24.1%
Issue	207/475=43.6%	127/313=40.6%
Stack	216/486=44.4%	123/320=38.4%
Issue + Stack	234/475=49.3%	149/313=47.6%

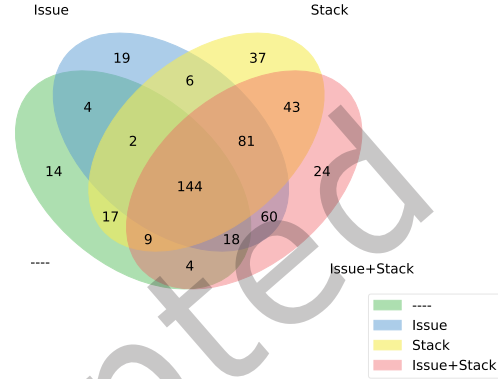


Fig. 7. Overlap of buggy method localization in Defects4J v2.0

Table 2 shows the effectiveness of buggy method localization in these four experiments. The first row, labeled “—”, indicates that only the related (executed) methods are fed to the LLM for buggy method localization. The second, third and fourth rows represent scenarios when, in addition to related method localization, issue content, error stack, or both are provided. Following the approach described in LLMAO [46], we consider a buggy method correctly localized if at least one match in the LLM’s response corresponds to the buggy method (single-method bugs) or one of the buggy methods (non-single-method bugs) in the ground truth. It is worth mentioning that there exist bug cases where certain software artifacts can not be extracted, and in this case we calculate the ratio by dividing only the number of bug cases with certain software artifacts information. Table 2 shows that, when issue content or error stack is provided, the LLM is able to localize more buggy methods than nothing is provided. When issue content is provided, 15.8% more single buggy methods and 16.5% more non-single buggy methods are correctly localized. When error stack is included, 16.6% more single buggy methods and 14.3% more non-single buggy methods are correctly localized. Furthermore, combining issue content and error stack achieves the best performance, which results in 49.3% in localizing single buggy methods and 47.6% of non-single buggy methods.

Figure 7 shows the overlap of results of buggy method localization with different artifacts in Defects4J v2.0. We can see that when provided with issue content, the LLM can localize 19 extra bugs correctly. Similarly, when provided with error stack, the LLM can localize 37 extra bugs correctly. This finding verifies our assumption in Section 2 that different kinds of software artifacts can complement each other in buggy method localization.

As existing SOTA fault localization approaches use Defects4J v1.2 dataset and adopt Top- n (the buggy method is in the top n list of the LLM’s response) to evaluate the performance of fault localization, we compare the performance of DEVLoRE with the SOTA approaches in the same dataset and adopt Top- n approach. Table 3 presents the comparison results of DEVLoRE with three state-of-the-art (SOTA) LLM-based approaches. The row below the names of these approaches lists the artifacts used by each approach. Among them, SoapFL [34] uses codebase, failed test methods, and error stack trace; AutoFL [18] uses failed test methods along with error stack trace; and Agentless [40] uses codebase and issue message. In contrast to these approaches, DEVLoRE utilizes

Table 3. Method-level FL comparison with SOTA baselines on Defects4J v1.2

Project	#Bugs	DEVLoRe*			SoapFL [34]			AutoFL [18]			Agentless [40]		
		codebase, issue content, failed tests, error stack			codebase, failed tests, error stack			failed tests, error stack			codebase, issue message		
		Top1	Top3	Top5	Top1	Top3	Top5	Top1	Top3	Top5	Top1	Top3	Top5
Chart	26	18	18	18	17	22	22	16	20	21	10	15	16
Lang	65	54	58	58	39	48	49	35	43	43	34	40	48
Math	106	81	86	86	57	71	73	53	67	70	45	51	52
Time	27	12	13	13	10	13	13	14	15	16	8	10	12
Mockito	38	22	22	22	20	21	22	13	17	17	21	26	26
Closure	133	26	37	41	32	57	57	18	24	24	18	31	31
Overall	395	213	234	238	175	232	236	149	186	191	136	173	185

*: To ensure a fair comparison, DEVLoRe uses the same GPT-3.5 model as the other baselines.

four artifacts: issue content, error stack, failed tests, and codebase. Failed tests and codebase are used to retrieve related methods via MethodRecorder. It is worth to mention that DEVLoRe also outperform the SOTA learning-based approaches such as GRACE [29] and FLUCCS [38]. But in this study we focused on how various artifacts can help LLMs with bug localization and program repair, so we did not include the comparison with non-LLM-based approaches. Since the other three baselines use the GPT-3.5 model, for a fair comparison, DEVLoRe also uses the same GPT-3.5 model. As we can see, DEVLoRe outperforms SoapFL, AutoFL and Agentless in Top-1, Top-3 and Top-5 metrics. The last row shows that, DEVLoRe outperforms other approaches in Top-1 and achieves similar performance in Top-3 and Top-5 with SoapFL, demonstrating its ability to precisely localize buggy methods.

5.1.2 Localizing Buggy Line(s). To evaluate LLMs' ability in localizing buggy lines, we provide the LLM with the buggy method(s) baseline (i.e., the ground truth which can be extracted from human developer's correct patch) and ask the LLM to predict which lines are buggy. If the line number generated by the LLM exactly matches the first line added in the human developer's correct patch in Defects4J, we consider it an exact match. If the predicted line number falls within a range of n lines from the first added line in the human developer's patch (for example, if the correct patch starts at Line 368, and the LLM's prediction falls within the range from $368 - n$ to $368 + n$), we classify it as a Range- n match.

Table 4 shows the effectiveness of buggy line localization with different artifacts. We can see that when the LLM is fed with issue content, it performs the best among all single software artifacts, achieving 49.5% in exact matches, 74.4% in Range-3, and 77.5% in Range-5 for single methods. When adding more software artifacts, the performance does not appear to improve. For example, combining issue content with the error stack does not yield better exact, Range-3, or Range-5 buggy line matches for single methods compared to using issue content alone. However, as seen from Figure 8, the combination does successfully localize 3 additional buggy lines that cannot be identified using either issue content or the error stack individually. The last row indicates that, when combining all artifacts, 68.9% of buggy lines in single-method bugs and 9% of buggy lines in non-single-method bugs are correctly localized. Similar to Figure 7, Figure 8 demonstrates that, while most of the correctly localized buggy lines across different artifacts overlap, different combinations of artifacts each have their own advantages (since only five areas can be displayed in the overlap figure, we randomly select five categories for visualization). For example, using only the issue content can correctly localize 7 more buggy lines, while using only the debug information can localize 15 more.

Table 4. Effectiveness of buggy line localization with different software artifacts

	Match	Single Method	Non-Single
—	Exact	40.3%	4.3%
	Range-3	68.1%	8.1%
	Range-5	72.8%	9.8%
Issue	Exact	49.5%	5.2%
	Range-3	74.4%	9.2%
	Range-5	77.5%	11.0%
Stack	Exact	45.0%	4.9%
	Range-3	66.9%	9.0%
	Range-5	71.0%	10.1%
Debug	Exact	39.9%	3.5%
	Range-3	65.4%	6.4%
	Range-5	70.3%	8.1%
Stack + Debug	Exact	44.8%	5.2%
	Range-3	66.9%	9.0%
	Range-5	70.3%	10.7%
Issue + Debug	Exact	50.1%	5.2%
	Range-3	70.3%	8.4%
	Range-5	73.2%	9.8%
Issue + Stack	Exact	49.1%	5.5%
	Range-3	73.2%	10.1%
	Range-5	77.5%	11.8%
Issue + Stack + Debug	Exact	50.3%	5.2%
	Range-3	71.4%	11.3%
	Range-5	73.8%	12.7%
Union	Exact	68.9%	9.0%
	Range-3	83.8%	16.5%
	Range-5	85.7%	19.1%

Answer to RQ1: Among all three types of single software artifacts, issue content is the most effective in assisting the LLM with bug localization. Furthermore, different types of software artifacts complement each other in localizing buggy methods and buggy lines. This aligns with how human developers typically approach bug localization, as they often rely on information from multiple sources to identify bugs.

5.2 RQ2. Program Repair Based on Provided Method-level Fault Localization

To evaluate the effectiveness of program repair, we first extract buggy methods from human’s correct patches and used these buggy methods as the baseline. Then we use DebugRecorder to get the debug information in these buggy methods. Finally, we provide the LLM with different artifacts and check how many plausible patches (passing all tests) it could generate.

Table 5 shows that, error stack achieves the best performance in generating plausible patches for single-method bugs (27.0%) among all three software artifacts, while issue content performs the best for non-single buggy methods. If feeding the LLM with all three types of software artifacts, we can get the best performance,

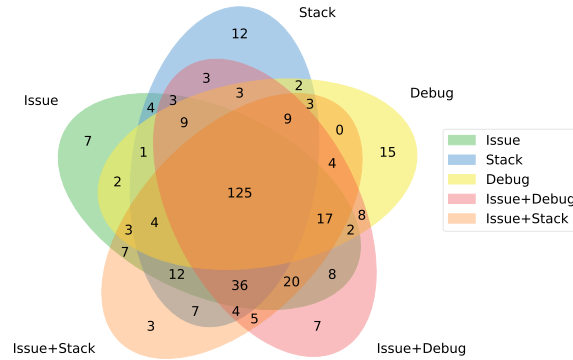


Fig. 8. Buggy line localization exact match with different artifacts in Defects4J v2.0

	Single Method	Non Single
—	46/489=9.4%	4/345=1.2%
Issue	127/478=26.6%	14/337=4.2%
Stack	132/489=27.0%	13/345=3.8%
Debug	114/468=24.4%	11/326=3.4%
Issue+Stack	159/464=34.3%	21/328=6.4%
Issue+Debug	191/457=41.8%	24/318=7.6%
Stack+Debug	139/462=30.1%	14/321=4.4%
Issue+Stack+Debug	197/457=43.1%	30/318=9.4%
Union	274/489=56.0%	50/346=14.5%

which is 43.1% and 9.4% in patching single-method bugs and non-single-method bugs, respectively. Figure 9 shows that different artifacts complement each other in program repair. Error stack can help the LLM fix 17 extra bugs while issue content can fix 13 extra bugs. Combining issue content with debug info can assist the LLM in fixing 23 extra bugs.

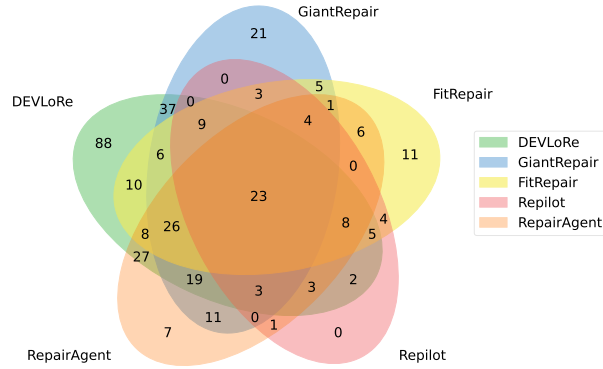


Fig. 10. A Comparison between DEVLoRe with other LLM-based SOTA program repair approaches

the other five LLM-based approaches. In total, DEVLoRe can fix 274 buggy methods, which is 60.2% more than GiantRepair [24]. Furthermore, Figure 10 shows that 88 bugs can only be fixed by DEVLoRe. This result demonstrates DEVLoRe’s ability in fixing more and extra bugs by leveraging different types of software artifacts.

Table 6. Plausible patches with provided method-level fault localization in **single buggy method** projects

Project	#Bugs	DEVLoRe	GaintRe- pair	RepairAgent	ChatRepair	FitRepair	Repilot
		buggy method, issue content, debugging info, stack traces	buggy method	failed tests, all methods in codebase	failed tests, buggy lines	buggy lines within buggy method	buggy method
Chart	16	12	8	11	15	8	6
Closure	95	37	32	27	37	29	21
Lang	42	32	14	17	21	17	15
Math	74	53	26	29	32	23	20
Time	16	3	1	2	6	3	2
Mockito	16	10	6	6	3	4	0
Defects4J v1.2	259	142	87	74	114	85	64
Defects4J v2.0	230	132	84	90	48	44	47
Total	489	274	171	164	162	129	111

Answer to RQ2: Among all three types of single software artifacts, error stack is the most effective in assisting the LLM with program repair with provided buggy method localization. Different combination of software artifacts complement each other in generating plausible patches and combining all three artifacts can achieve 56.0% of program repair, outperforming the SOTA approaches in program repair.

5.3 RQ3. End-to-end Performance from Code Repository to Program Repair

Currently, most of LLM-based approaches focus on either fault localization or program repair based on the already-localized buggy methods. We investigated RQ1 and RQ2 to compare with these approaches. In contrast, our DEVLoRE fully relies on LLMs for both fault localization and program repair. Furthermore, from fault localization to program repair, we not only provide the LLM with the localized buggy method signatures and lines, but also supply it with the complete method body. When the LLM is asked for the program repair task, it may reconsider and repair other parts of the code beyond the specified buggy location. Therefore, the overall end-to-end performance from the buggy code without localization to a complete program repair cannot be simply calculated by multiplying the fault localization rate by the program repair rate. Conducting a comprehensive end-to-end assessment can offer a better understanding of DEVLoRE's performance throughout the entire fault localization and program repair process.

Table 7 shows DEVLoRE's end-to-end performance with different software artifacts. When using issue content alone, the LLM can repair 21.8% of bugs, which is the highest among all single software artifacts. This result is also consistent with Agentless [40], one of the best approaches in SWE-bench lite (with Python projects) [4], which relies solely on issue descriptions to assist LLMs in resolving GitHub issues.

Furthermore, by combining issue content with debugging information and the error stack, the LLM can fix an additional 1.2% and 4.0% of single-method bugs, respectively. When all three software artifacts are combined, the LLM can repair 28.0% of single-method bugs and 11.2% of non-single-method bugs. Furthermore, the combination of different artifacts achieves an end-to-end 39.7% fix rate for single-method bugs and 17.1% for non-single-method bugs. Figure 11 shows similar observations as other two RQs, that different combinations of software artifacts can complement each other in the end-to-end process of bug localization and program repair.

Table 7. End-to-end plausible patch in Defects4J v2.0

	Single Method	Non Single
—	23/488=4.7%	6/346=1.7%
Issue	103/473=21.8%	30/335=9.0%
Debug	38/413=9.2%	3/295=1.0%
Stack	83/488=17.0%	14/346=4.0%
Issue+Debug	92/400=23.0%	30/283=10.6%
Issue+Stack	123/477=25.8%	31/338=9.2%
Stack+Debug	73/413=17.7%	13/295=4.4%
Issue+Stack+Debug	114/407=28.0%	33/295=11.2%
Union	194/489=39.7%	59/346=17.1%

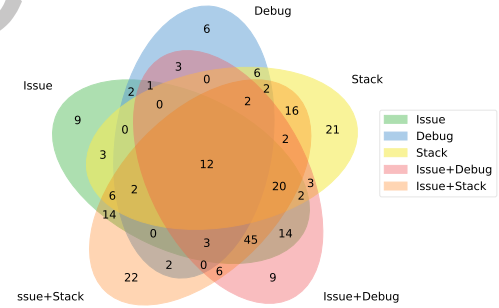


Fig. 11. Overlap of the overall performance in Defects4J v2.0

We divide the time spent on each process by the number of fixed bugs to calculate the average time required to fix a bug. Table 8 shows the average time spent to fix a bug in each process of DEVLoRE using our hardware device (Intel(R) Xeon(R) Platinum 8352V CPU 2.10GHz, 120GB RAM). We observe that the entire localization and repair process takes approximately 15 seconds, with the evaluation of patches being the most time-consuming step, as it involves running all relevant tests. Additionally, since we select the most cost-efficient GPT-4o-mini model, the average cost to fix a bug is **\$0.057**. We calculate this by dividing the total cost of using the LLM when issue content, error stack, and debug information are provided by the number of plausible patches generated, which is 147.

Table 8. Time spent on each process of DEVLoRE

Process	Time Spent (On Average)
Extract Related Methods by MethodRecorder	≈ 5s
Localize Buggy Methods by LLM	<1s
Extract Debugging Information by DebugRecorder	≈ 5s
Localize Buggy Lines by LLM	<2s
Generate Patches by LLM	<2s
Evaluate Plausible Patches by the Defects4J framework	≈ 300s
Total	≈ 315s

Using our DEVLoRE framework for this end-to-end software maintenance activity, 253 (194 for single buggy methods and 59 for non-single buggy methods) plausible patches can be generated. To the best of our knowledge, DEVLoRE outperforms all current SOTA end-to-end approaches in the Defects4J dataset (the plausible rate of Toggle [12] is 24.2% for 58/240 single-hunk bugs and the plausible rate of FixAgent [23] without Web search engine is 245/835=29.3% at a cost of \$0.364 per bug).

Answer to RQ3: Among the three types of software artifacts, issue content is the most effective in assisting the LLM throughout the end-to-end process, from code repository analysis to fault localization and program repair. Different combinations of software artifacts complement each other, enhancing the overall bug localization and repair process. Moreover, our DEVLoRE framework can effectively fix bugs at a low cost and within a reasonable time frame.

6 DISCUSSION

6.1 Analysis of Results

6.1.1 A simple but efficient framework: Unlike approaches that rely on patch skeletons [24], fix templates [50], or type checking [53], DEVLoRE adopts a straightforward framework that allows LLMs to handle both fault localization and program repair with the aid of two lightweight tools we implemented MethodRecorder and DebugRecorder, making it both simple and efficient. The findings of RQ1, RQ2, and RQ3 demonstrate that different software artifacts may lead to different performance in bug localization or program repair. Also, because there is no constraints such as fill-in-the-blank templates or code skeletons for generating patches [24, 50], DEVLoRE can fix bugs that other tools cannot, especially some non-single method bugs. Our results in Table 7 show that by combining issue content, stack error, and debug information, our approach can fix 11.2% of non-single method bugs. More importantly, combinations of software artifacts can achieve the best performance in all experiments: bug localization, program repair, and the overall streamlined process. One explanation for the strong performance of DEVLoRE is that by integrating multiple sources of information, the noise inherent in any single source can be significantly reduced [31]. This allows the LLM to focus on the consistent and complementary aspects of the different artifacts, enabling it to perform more effective reasoning and making better use of the available data. While it may be argued that the good results of DEVLoRE are due to the use of GPT-4o models, a recent study shows that directly using the GPT-4o model does not improve the fix rate [24]. Therefore, we argue that our proposed framework is the key to achieving the high rate in fault localization and program repair.

6.1.2 A strict input/output prompt design: Table 1 presents the prompts used in our DEVLoRE framework. In the GENERAL TASK PROMPT, the LLM is asked to act as a software engineer and conduct the review process, helping the LLM form a clear understanding of the overall task. The INPUT PROMPT includes various types

of software artifacts, with clear symbols denoting different hierarchy levels and structures. For example, the `{related_methods}` in the input prompt wraps the class names in `###` symbols, and the method signatures are separated by line breaks. Also, the first line of `{debugging_info}` in the input prompt represents the currently executed method line, and the second line represents the names and values of the variables in current context (e.g., `commons.lang3.math.NumberUtils:createNumber:468{hexDigits:8}` represents that the code is about to execute Line 468 and the value of the local variable `hexDigits` is 8). These strict formatting specifications helps the LLM “understand” the structure of the various information from different software artifacts. The EXPECTED OUTPUT PROMPT is very strict in DEVLoRE. When localizing buggy methods, we ask the LLM to return a set of buggy method or field locations in the format `path.to.ClassA::methodA`. For buggy lines, we request the LLM to return a set of buggy line locations in the format `path.to.ClassA line:20`. During program repair, we employ the well-known *SEARCH/REPLACE* method, which is commonly used in many state-of-the-art program repair approaches [27, 32, 40, 51]. By enforcing this strict output format, it significantly reduces the likelihood of hallucinations from the LLM. We believe that the clear and strict prompt design in our DEVLoRE framework has helped the LLM achieve strong performance in fault localization and program repair.

Table 9. A comparison from plausible patch to correct patch in Defects4J v1.2

Project	DEVLoRE	GaintRepair [24]	Tare [53]	TBar [26]
Chart	7/12	7/10	11/14	7/10
Closure	10/16	16/33	12/23	6/10
Lang	8/16	12/19	12/19	4/11
Math	28/48	22/40	18/34	12/26
Time	0/2	1/3	2/3	1/2
Mockito	6/11	6/6	2/2	1/2
Total	59/105	64/111	57/95	31/61
P(%)	56.2%	57.7%	60.0%	50.8%

* X/Y denotes X correct patches and Y plausible patches.

6.1.3 Plausible patch to correct patch: Most state-of-the-art (SOTA) program repair approaches use plausible patches that pass all unit tests as an important evaluation metric, since generating plausible patches within limited time and resources is crucial for practical applications. To facilitate a better comparison, we also used plausible patches in RQ2 (Which software artifacts can better assist LLMs in generating valid patches when provided with buggy methods?) and RQ3 (What is the overall performance of DEVLoRE in bug localization and program repair?). Some may argue that a high number of plausible patches does not necessarily translate to a high number of semantically correct patches. To address this, we manually inspected the plausible patches, using the approach described in Section 3.4. Table 9 compares the end-to-end repair results on Defects4J v1.2, as all the baselines consistently used this benchmark. Our DEVLoRE framework uses the LLM to localize buggy methods and lines, whereas all three approaches – GiantRepair [24], Tare [53], and TBar [26] – employ the spectrum-based algorithm Ochiai [5], implemented by GZoltar [7], to localize buggy methods. As shown in Table 9, the ratio of correct to plausible patches (56.2%) produced by our DEVLoRE is lower than that from Tare (60.0%) and from GiantRepair (57.7%). This difference is understandable, as GiantRepair enforces an AST patch skeleton and Tare uses a typing-checking mechanism. Notably, 59 out of 105 plausible patches generated by DEVLoRE are evaluated as correct patches, which is higher than the 57 in Tare and 31 in TBar. This finding demonstrates DEVLoRE’s ability to generate candidate patches that not only pass all unit tests but are also semantically correct.

Table 10. Resolution rates of frameworks using a GPT-4o model on the SWE-bench Lite Leaderboards

Tool	LLM	Resolved/Selected	%Resolved
DEVLoRE	GPT-4o-mini	46/207	22.2%
SWE-agent [49]	GPT-4o	55/300	18.3%
AppMap Navie [2]	GPT-4o	65/300	21.7%
Aider [1]	GPT-4o+Claude-3	79/300	26.3%
Agentless [40]	GPT-4o	82/300	27.3%
MarsCode [27]	Multiple LLMs(includes GPT-4o)	118/300	39.3%

6.2 Comparison with Baselines of Different Models or on Different Languages

6.2.1 Comparison with Baselines of Different Models. We selected state-of-the-art LLM-based fault localization and program repair frameworks on the Defects4J dataset as our baselines. One may argue that some baselines do not use a GPT-4o model, and that the superior performance of our DEVLoRE framework is due to the use of a more powerful model. However, as shown in Table 3, even using the same GPT-3.5 model as the baselines, our framework outperforms existing state-of-the-art fault localization approaches. For program repair, the models used in these baselines vary significantly. For example, GiantRepair and Repilot combine the outputs of multiple LLMs, while FitRepair fine-tunes CodeT5 using 50% of the Defects4J dataset. As a result, it is challenging to align all baselines under a single model. According to a recent study [8], GPT-4o achieves only 16.4% and 27.8% accuracy at pass@1 and pass@5, respectively, in generating correct patches given a buggy function on the Defects4J dataset. Additionally, as noted in the discussion section of the GiantRepair paper [24], given a buggy function, GPT-4o-mini achieves 21.7% accuracy when generating 200 patches per bug on the Defects4J dataset (pass@200). In contrast, DEVLoRE, which leverages issue content, error stack and debug information, achieves 56.0% accuracy in program repair using GPT-4o-mini (pass@30). We believe that this performance gap is due to our framework’s ability to effectively leverage various software artifacts.

6.2.2 Comparison with Baselines on Different Programming Languages. Our framework was largely inspired by Agentless. Initially, we attempted to apply Agentless to the Defects4J dataset, but the resolution rate was below 10%. This led us to consider incorporating debugging information and re-implementing Agentless for evaluation on the Defects4J dataset. During this process, we encountered a major issue: Agentless uses only the skeleton of the source files during the fault localization step, without incorporating the detailed source code. To address this limitation and incorporate both debugging information and actual code lines, we extended the original Agentless framework to include the full source code as input to the LLM. However, this introduced a new challenge: the input frequently exceeded the LLM’s token limit. To overcome this, we introduced MethodRecorder, which identifies and retains only the methods relevant to the failing test cases, thereby reducing the number of code elements passed to the LLM. We also pruned the debug information by recording only variables that had changed and cropping long arrays and lists to further reduce the token size. Without such adjustments or modifications to Agentless—even when providing all the artifacts listed in this paper—its end-to-end performance remained below 15%. This demonstrates that directly applying a leading program repair framework designed for Python code to Java code does not yield competitive results.

To compare with leading frameworks and evaluate DEVLoRE’s capability on Python code, we re-implemented our framework and tested it on SWE-bench Lite. The first challenge we encountered was that, even after filtering with MethodRecorder, the length of the related methods for one-third of the bugs in SWE-bench Lite still

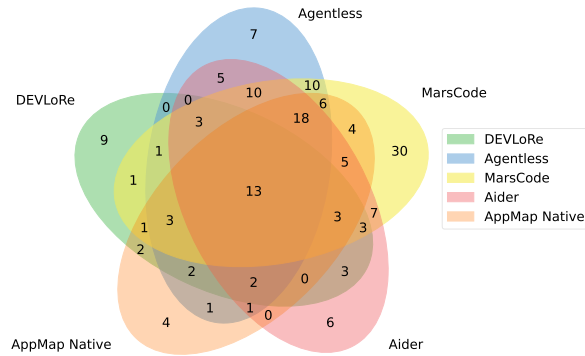


Fig. 12. A comparison between DEVLoRe with baselines on SWE-bench Lite

exceeded the LLM’s token limit. As a result, we were only able to provide related methods to the LLM for 207 out of 300 bugs in SWE-bench Lite. Additionally, we observed that Python’s debugging information is significantly more verbose than that of Java. For some bugs, the required patches involve modifying top-level constructs such as imports or global variables, for which debugging information cannot be extracted by executing the test cases. As shown in Table 10, among the 207 bugs with related methods information, DEVLoRe successfully fixed 46 ($46/207=22.2\%$). We also provide a comparison with the SWE-bench Lite Leaderboards using the GPT-4o model. The results in Table 10 shows that DEVLoRe did not outperform Agentless or SWE-agent or MarsCode in the total number of fixed Python bugs. We suspect this is due to fundamental differences in coding practices between Java and Python, which result in related methods or debugging information being unavailable for a portion of the issues. Moreover, we compared the resolved issue IDs of DEVLoRe with those of baselines. The overlap of resolved issues among the baselines and DEVLoRe in Figure 12 shows that DEVLoRe was able to resolve 9 unique issues, despite resolving only 46 issues in total. Manual inspection of these 9 issues revealed that all of them involved the use of debugging information. This result suggests that the debugging information automatically extracted in our framework—among these baselines, only AppMap Naive utilizes runtime data when available in an interactive way—shows promise in addressing a subset of issues. Moreover, this experiment highlights that directly applying a leading framework from SWE-bench to Defects4J, or vice versa, may not yield state-of-the-art results. Generalizing the framework across different programming languages will be part of our future work.

6.3 Threats to Validity

The first threat to validity is the accuracy of the two tools we implemented: MethodRecorder and DebugRecorder. Both tools rely on mature Java agent technology [3]. We randomly selected several projects and manually verified the outputs of both tools. The manual inspection showed that the outputs were accurate. However, we did not inspect all projects, which may pose a threat to the construct validity.

Another potential threat is that the GPT-4o-mini model used in this work may have been trained on open-source projects from GitHub, which could overlap with the Defects4J dataset, leading to possible data leakage. To mitigate this, we also randomly selected 100 bugs from another dataset, GrowingBugs [16], and found the plausible fixing rate to be 39%, which may help alleviate this concern. Additionally, the debugging information

requires dynamic analysis, which is unlikely to have been used during the model’s training. Also as found in [8, 24], directly using the GPT-4o model can not improve the fix rate.

The final threat to external validity is that our systematic experiments used only one model and one approach, although using GPT-3.5 in DEVLoRE also outperform SOTA fault localization approaches. But the findings may not generalize to other models or approaches. We plan to explore this in our future work. Also, as discussed in Section 6.2, generalizing our framework to other programming languages, such as Python, does not currently achieve state-of-the-art performance. We plan to further improve and evaluate DEVLoRE on additional datasets across multiple programming languages in our future work.

7 RELATED WORK

7.1 Large Language Models for Fault Localization

Recently, there has been significant interest in using LLMs for fault localization. Toggle incorporated additional contextual information, such as the buggy line number or code review comments, and greatly enhances the accuracy of predicting both the starting and ending buggy tokens [12]. AGENTFL employs a multi-agent system based on ChatGPT and frames the fault localization task as a three-step process: comprehension, navigation, and confirmation. In each step, AGENTFL deploys specialized agents, each with unique expertise, and uses different tools to address specific tasks [34]. CrashTracker conducts static analysis to map each crash to the corresponding exception instance and identify potential buggy candidates. It then utilizes LLMs to enhance the explainability of the localization results [45]. Jiang et al. assessed the performance of recent commercial closed-source general-purpose LLMs, such as ChatGPT 3.5, ERNIE Bot 3.5, and iFlytek Spark 2.0, on line-level fault localization with the provided buggy method [15]. LLMA0 fine-tunes LLMs with 350M, 6B, and 16B parameters on small, curated corpora like Defects4J, improving Top-1 fault localization by 2.3%-54.4% and Top-5 results by 14.4%-35.6%, compared to the state-of-the-art machine learning fault localization [46]. AutoFL prompts an LLM to use function calls for navigating a repository, enabling effective fault localization in large codebases while overcoming the LLM context length limit. It also generates an explanation of the bug and suggests a fault location [18]. LLM4FL combines traditional spectrum-based fault localization with prompt chaining to divide large coverage data into manageable groups. By employing multiple LLM agents, it navigates the codebase more effectively to localize faults [35]. Like LLM4FL and AutoFL which combines transitional fault localization tools, DEVLoRE incorporates a static code analysis tool, MethodRecorder, to identify relevant buggy methods in a lightweight manner by invoking failing tests. However, unlike the above approaches, we provide the LLM with a combination of software artifacts, including issue content, error stack traces, and debugging information, which are commonly used by human developers for fault localization. This enables DEVLoRE to outperform state-of-the-art fault localization methods with greater efficiency and lower cost.

7.2 Large Language Models for Program Repair

Recent studies have extensively explored the use of LLMs for program repair. ChatRepair initially provides the LLM with relevant test failure information and then learns from both the failures and successes of previous patching attempts for the same bug, enhancing its ability for more effective APR [44]. GAMMA converts various fix templates into mask patterns and leverages a pre-trained language model to predict the correct code for the masked portions, treating APR as a fill-in-the-blank task [50]. Repilot generates a candidate patch by combining LLM suggestions with a Completion Engine, removing infeasible tokens and filling in gaps proactively [39]. FitRepair integrates the plastic surgery hypothesis into LLM-based APR, combining the direct use of LLMs with two domain-specific fine-tuning strategies and one prompting strategy to enhance its repair capabilities. It can directly generate the correct code in context, effectively “filling in the blanks” of missing code lines or hunks [41]. Tare incorporated type checking into neural program repair model and can successfully repair 62 and 32 bugs

from Defects4J v1.2 and Defects4J v2.0 [53]. GiantRepair creates patch skeletons from LLM-generated patches to narrow the patch space, then generates context-aware, high-quality patches by instantiating these skeletons for specific programs [24]. FixAgent unifies debugging through multi-agent collaboration and achieves strong performance with a three-layer hierarchical structure, where the final layer involves the use of a Web search engine [23]. MOREpair fine-tunes LLMs for program repair by adapting both to the syntactic nuances of code transformation and the underlying logic of code changes, enabling the generation of high-quality patches [48]. To leverage LLMs' capabilities and augmented information, CREF is a semi-automatic repair framework for programming tutors, highlighting the potential for enhancing LLMs' repair capabilities through tutor interactions and historical conversations [47]. RepairLLaMA is an innovative program repair method that finds optimal code representations for APR using fine-tuned models, and introduces a state-of-the-art parameter-efficient fine-tuning technique (PEFT) for program repair [36]. Our approach differs from the aforementioned approaches in two key ways. First, we do not rely on fill-in-the-blank templates or skeletons for generating patches, which helps avoid patch overfitting problems in program repair [10, 20, 37]. Second, our fault localization step allows for the identification of multiple buggy methods, which are then fed into the LLM's repair process along with various software artifacts. This provides flexibility to address not only single-method bugs but also bugs spanning across different methods. As seen in Table 7, the end-to-end process from localization to repair can generate plausible patches for 17.1% of non-single-method bugs, while most existing program repair approaches primarily target single-method bugs. A recent study [33] investigated how to identify an optimal set of facts to maximize the performance of LLM-based program repair. Their framework adopts chain-of-thought prompts, uses artifacts from both the fixed (correct) and buggy programs, and is evaluated on the BugsInPy dataset. Complementary to their work, we show that extracting all artifacts solely from the buggy version—which can be found in most development scenarios—and using direct prompts can still yield effective results on both the Defects4J dataset and SWE-bench Lite. Moreover, our paper demonstrates that different combinations of artifacts can enhance one another, helping LLMs on both fault localization and program repair.

8 CONCLUSIONS AND FUTURE WORK

This paper presents an LLM-based framework, DEVLoRE, for streamlining fault localization and program repair. By mimicking human developers in addressing bug problems and integrating three different software artifacts, DEVLoRE demonstrates strong performance in both fault localization and program repair, outperforming current state-of-the-art approaches in terms of bug fixing rate, time, and cost on the Defects4J dataset. In addition, unlike more rigid approaches that ask LLMs to fill in the blank within a single buggy method or use the fix template, DEVLoRE feed the LLM with only different software artifacts and there are no constraints on the DEVLoRE framework regarding how it repairs bugs, DEVLoRE has shown significant potential in handling bugs that span across multiple methods.

Our future work will focus on the following directions: First, expanding DEVLoRE to support additional programming languages such as Python and C/C++, and evaluating and improving its effectiveness on projects written in these languages. Second, testing the DEVLoRE framework with a broader range of software artifacts—such as commit history, code review comments, and user documentation—to assess how these additional data sources can further enhance the bug localization and program repair process. Third, conducting a systematic evaluation involving multiple approaches and LLMs to better assess DEVLoRE's generalization capabilities.

DATA AVAILABILITY

The source code and experimental results of this work for replication are available at <https://github.com/XYZboom/DEVLoRE>.

ACKNOWLEDGMENTS

This work has been partially supported by the National Natural Science Foundation of China (NSFC) with Grant No. 62172311 and the Major Science and Technology Project of Hubei Province under Grant No. 2024BAA008. The numerical calculations in this paper have been done on the supercomputing system in the Supercomputing Center of Wuhan University.

REFERENCES

- [1] [n. d.]. Aider. <https://aider.chat>.
- [2] [n. d.]. AppMap Navie. <https://appmap.io/product/appmap-navie.html>.
- [3] [n. d.]. Instrumentation documents in Java SE 8. <https://docs.oracle.com/javase/8/docs/api/java/lang/instrument/Instrumentation.html>
- [4] 2024. SWE-bench Lite. <https://www.swebench.com/lite.html>.
- [5] Rui Abreu, Peter Zoetewij, and Arjan JC Van Gemund. 2007. On the accuracy of spectrum-based fault localization. In *Proceedings of the Testing: Academic and Industrial Conference Practice and Research Techniques-MUTATION (TAICPART-MUTATION)*. ACM, 89–98.
- [6] Islem Bouzenia, Premkumar Devanbu, and Michael Pradel. 2024. RepairAgent: An Autonomous, LLM-Based Agent for Program Repair. *arXiv preprint arXiv:2403.17134* (2024).
- [7] José Campos, André Ribeiro, Alexandre Perez, and Rui Abreu. 2012. Gzoltar: an eclipse plug-in for testing and debugging. In *Proceedings of the 27th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 378–381.
- [8] Viola Campos, Ridwan Shariffdeen, Adrian Ulges, and Yannic Noller. 2025. Empirical Evaluation of Generalizable Automated Program Repair with Large Language Models. *arXiv preprint arXiv:2506.03283* (2025).
- [9] Zimin Chen, Steve Kommrusch, Michele Tufano, Louis-Noël Pouchet, Denys Poshyvanyk, and Martin Monperrus. 2019. SequenceR: Sequence-to-Sequence Learning for End-to-End Program Repair. *IEEE Transactions on Software Engineering* 47, 9 (2019), 1943–1959.
- [10] Zhiwei Fei, Jidong Ge, Chuanyi Li, Tianqi Wang, Yuning Li, Haodong Zhang, LiGuo Huang, and Bin Luo. 2025. Patch Correctness Assessment: A Survey. *ACM Transactions on Software Engineering and Methodology* 34, 2 (2025), Article No.: 55.
- [11] Ali Ghanbari, Samuel Benton, and Lingming Zhang. 2019. Practical program repair via bytecode mutation. In *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. ACM, 19–30.
- [12] Soneya Binta Hossain, Nan Jiang, Qiang Zhou, Xiaopeng Li, Wen-Hao Chiang, Yingjun Lyu, Hoan Nguyen, and Omer Tripp. 2024. A Deep Dive into Large Language Models for Automated Bug Localization and Repair. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 1471–1493.
- [13] Nan Jiang, Kevin Liu, Thibaud Lutellier, and Lin Tan. 2023. Impact of code language models on automated program repair. In *Proceedings of the 45th IEEE/ACM International Conference on Software Engineering (ICSE)*. IEEE, 1430–1442.
- [14] Nan Jiang, Thibaud Lutellier, and Lin Tan. 2021. Cure: Code-aware neural machine translation for automatic program repair. In *Proceedings of the 43rd IEEE/ACM International Conference on Software Engineering (ICSE)*. IEEE, 1161–1173.
- [15] Shengbei Jiang, Jiabao Zhang, Wei Chen, Bo Wang, Jianyi Zhou, and Jie Zhang. 2024. Evaluating Fault Localization and Program Repair Capabilities of Existing Closed-Source General-Purpose LLMs. In *Proceedings of the 1st International Workshop on Large Language Models for Code (LLM4Code)*. ACM, 75–78.
- [16] Yanjie Jiang, Hui Liu, Xiaoqing Luo, Zhihao Zhu, Xiaye Chi, Nan Niu, Yuxia Zhang, Yamin Hu, Pan Bian, and Lu Zhang. 2022. Bugbuilder: An automated approach to building bug repository. *IEEE Transactions on Software Engineering* 49, 4 (2022), 1443–1463.
- [17] René Just, Darioush Jalali, and Michael D Ernst. 2014. Defects4J: A database of existing faults to enable controlled testing studies for Java programs. In *Proceedings of the 23rd International Symposium on Software Testing and Analysis (ISSTA)*. ACM, 437–440.
- [18] Sungmin Kang, Gabin An, and Shin Yoo. 2024. A quantitative and qualitative evaluation of LLM-based explainable fault localization. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 1424–1446.
- [19] Xuan Bach D. Le, David Lo, and Claire Le Goues. 2016. History driven program repair. In *Proceedings of the 23rd IEEE International Conference on Software Analysis, Evolution, and Reengineering (SANER)*. IEEE, 213–224.
- [20] Xuan-Bach D Le, Ferdian Thung, David Lo, and Claire Le Goues. 2018. Overfitting in semantics-based automated program repair. In *Proceedings of the 40th International Conference on Software Engineering (ICSE)*. ACM, 163–163.
- [21] Claire Le Goues, ThanhVu Nguyen, Stephanie Forrest, and Westley Weimer. 2011. GenProg: A Generic Method for Automatic Software Repair. *IEEE Transactions on Software Engineering* 38, 1 (2011), 54–72.
- [22] Claire Le Goues, Michael Pradel, and Abhik Roychoudhury. 2019. Automated program repair. *Commun. ACM* 62, 12 (2019), 56–65.
- [23] Cheryl Lee, Chunqiu Steven Xia, Longji Yang, Jen-tse Huang, Zhouxiu Zhu, Lingming Zhang, and Michael R Lyu. 2024. A unified debugging approach via llm-based multi-agent synergy. *arXiv preprint arXiv:2404.17153* (2024).
- [24] Fengjie Li, Jiajun Jiang, Jiajun Sun, and Hongyu Zhang. 2025. Hybrid automated program repair by combining large language models and program analysis. *ACM Transactions on Software Engineering and Methodology* (2025).

- [25] Yi Li, Shaohua Wang, and Tien N. Nguyen. 2020. DLFix: context-based code transformation learning for automated program repair. In *Proceedings of the 42nd ACM/IEEE International Conference on Software Engineering (ICSE)*. ACM, 602–614.
- [26] Kui Liu, Anil Koyuncu, Dongsun Kim, and Tegawendé F. Bissyandé. 2019. TBar: revisiting template-based automated program repair. In *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. ACM, 31–42.
- [27] Yizhou Liu, Pengfei Gao, Xinchun Wang, Jie Liu, Yexuan Shi, Zhao Zhang, and Chao Peng. 2024. MarsCode Agent: AI-native Automated Bug Fixing. *arXiv preprint arXiv:2409.00899* (2024).
- [28] Fan Long and Martin Rinard. 2015. Staged program repair with condition synthesis. In *Proceedings of the 10th Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*. ACM, 166–178.
- [29] Yiling Lou, Qihao Zhu, Jinhao Dong, Xia Li, Zeyu Sun, Dan Hao, Lu Zhang, and Lingming Zhang. 2021. Boosting coverage-based fault localization via graph-based representation learning. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. ACM, 664–676.
- [30] Sergey Mechtaev, Jooyong Yi, and Abhik Roychoudhury. 2016. Angelix: scalable multiline program patch synthesis via symbolic analysis. In *Proceedings of the 38th International Conference on Software Engineering (ICSE)*. ACM, 691–701.
- [31] Xiangxin Meng, Zexiong Ma, Pengfei Gao, and Chao Peng. 2024. An Empirical Study on LLM-based Agents for Automated Bug Fixing. *arXiv preprint arXiv:2411.10213* (2024).
- [32] Siru Ouyang, Wenhao Yu, Kaixin Ma, Zilin Xiao, Zhihan Zhang, Mengzhao Jia, Jiawei Han, Hongming Zhang, and Dong Yu. 2024. RepoGraph: Enhancing AI Software Engineering with Repository-level Code Graph. *arXiv preprint arXiv:2410.14684* (2024).
- [33] Nikhil Parasaram, Huijie Yan, Boyu Yang, Zineb Flahy, Abriele Qudsi, Damian Ziaber, Earl Barr, and Sergey Mechtaev. 2024. The fact selection problem in LLM-based program repair. *arXiv preprint arXiv:2404.05520* (2024).
- [34] Yihao Qin, Shangwen Wang, Yiling Lou, Jinhao Dong, Kaixin Wang, Xiaoling Li, and Xiaoguang Mao. 2025. SoapFL: A Standard Operating Procedure for LLM-based Method-Level Fault Localization. *IEEE Transactions on Software Engineering* 51, 4 (2025), 1173–1187.
- [35] Md Nakhla Rafi, Dong Jae Kim, Tse-Hsun Chen, and Shaowei Wang. 2024. Enhancing Fault Localization Through Ordered Code Analysis with LLM Agents and Self-Reflection. *arXiv preprint arXiv:2409.13642* (2024).
- [36] André Silva, Sen Fang, and Martin Monperrus. 2023. Repairllama: Efficient representations and fine-tuned adapters for program repair. *arXiv preprint arXiv:2312.15698* (2023).
- [37] Edward K Smith, Earl T Barr, Claire Le Goues, and Yuriy Brun. 2015. Is the cure worse than the disease? overfitting in automated program repair. In *Proceedings of the 10th Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*. ACM, 532–543.
- [38] Jeongju Sohn and Shin Yoo. 2017. Flucss: Using code and change metrics to improve fault localization. In *Proceedings of the 26th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. ACM, 273–283.
- [39] Yuxiang Wei, Chunqiu Steven Xia, and Lingming Zhang. 2023. Copiloting the copilots: Fusing large language models with completion engines for automated program repair. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. ACM, 172–184.
- [40] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. 2024. Agentless: Demystifying LLM-based Software Engineering Agents. *arXiv preprint arXiv:2407.01489* (2024).
- [41] Chunqiu Steven Xia, Yifeng Ding, and Lingming Zhang. 2023. The Plastic Surgery Hypothesis in the Era of Large Language Models. In *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 522–534.
- [42] Chunqiu Steven Xia, Matteo Paltenghi, Jia Le Tian, Michael Pradel, and Lingming Zhang. 2024. Fuzz4All: Universal Fuzzing with Large Language Models. *arXiv preprint arXiv:2308.04748* (2024).
- [43] Chunqiu Steven Xia, Yuxiang Wei, and Lingming Zhang. 2023. Automated program repair in the era of large pre-trained language models. In *Proceedings of the 45th IEEE/ACM International Conference on Software Engineering (ICSE)*. IEEE, 1482–1494.
- [44] Chunqiu Steven Xia and Lingming Zhang. 2023. Keep the Conversation Going: Fixing 162 out of 337 bugs for \$0.42 each using ChatGPT. *arXiv preprint arXiv:2304.00385* (2023).
- [45] Jiwei Yan, Jinhao Huang, Chunrong Fang, Jun Yan, and Jian Zhang. 2024. Better Debugging: Combining Static Analysis and LLMs for Explainable Crashing Fault Localization. *arXiv preprint arXiv:2408.12070* (2024).
- [46] Aidan Z. H. Yang, Claire Le Goues, Ruben Martins, and Vincent Hellendoorn. 2024. Large Language Models for Test-Free Fault Localization. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE)*. ACM, 1–12.
- [47] Boyang Yang, Haoye Tian, Weiguo Pian, Haoran Yu, Haitao Wang, Jacques Klein, Tegawendé F Bissyandé, and Shunfu Jin. 2024. Cref: An LLM-based conversational software repair framework for programming tutors. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. ACM, 882–894.
- [48] Boyang Yang, Haoye Tian, Jiadong Ren, Hongyu Zhang, Jacques Klein, Tegawendé F Bissyandé, Claire Le Goues, and Shunfu Jin. 2024. Multi-Objective Fine-Tuning for Enhanced Program Repair with LLMs. *arXiv preprint arXiv:2404.12636* (2024).
- [49] John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. Swe-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems* 37 (2024), 50528–50652.

- [50] Quanjun Zhang, Chunrong Fang, Tongke Zhang, Bowen Yu, Weisong Sun, and Zhenyu Chen. 2023. Gamma: Revisiting template-based automated program repair via mask prediction. In *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 535–547.
- [51] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. 2024. Autocoderover: Autonomous program improvement. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. ACM, 1592–1604.
- [52] Qihao Zhu, Zeyu Sun, Yuan-an Xiao, Wenjie Zhang, Kang Yuan, Yingfei Xiong, and Lu Zhang. 2021. A syntax-guided edit decoder for neural program repair. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. ACM, 341–353.
- [53] Qihao Zhu, Zeyu Sun, Wenjie Zhang, Yingfei Xiong, and Lu Zhang. 2023. Tare: Type-aware neural program repair. In *Proceedings of the 45th IEEE/ACM International Conference on Software Engineering (ICSE)*. IEEE, 1443–1455.

Just Accepted